CS2614 — Computer System Architecture

Chapter 2: Digital Components

Based on M. Morris Mano, Computer System Architecture, 3rd ed.

1 Integrated Circuits

Digital circuits are constructed with integrated circuits.

An integrated circuit, or IC, is a small semiconductor chip containing electronic components interconnected to perform one or more digital functions.

As integrated-circuit technology developed, progressively larger numbers of gates could be placed within a single package.

WHITEBOARD

IC = interconnected digital circuitry on one chip

SSI	several gates
MSI	approximately 10–200 gates
LSI	hundreds to thousands of gates
VLSI	thousands of gates and beyond

Integrated circuits are also classified according to their digital logic family.

Mano identifies TTL, ECL, MOS, and CMOS as major families.

TTL became a widely used standard family.

ECL is associated with high-speed operation.

MOS technology permits high component density.

CMOS is particularly useful when low power consumption is important.

WHITEBOARD

Digital Logic Families

TTL	widely used standard family
ECL	high speed
MOS	high component density
CMOS	low power consumption

The remainder of the chapter introduces digital components that serve as reusable building blocks in computer organization.

2 Decoders

A binary code of n bits can represent as many as

$$2^n$$

distinct combinations.

A decoder is a combinational circuit that converts an n-bit coded input into as many as 2^n distinct output lines.

A decoder with n inputs and m outputs is commonly called an n-to-m-line decoder, where

$$m \leq 2^n.$$

WHITEBOARD

$$\boxed{n\text{-to-}m \text{ decoder}, \qquad m \leq 2^n}$$

For the full decoding case:

$$\boxed{n \text{ inputs} \rightarrow 2^n \text{ outputs}}$$

2.1 3-to-8-Line Decoder

A 3-to-8-line decoder accepts three binary inputs and produces eight output lines.

Each output corresponds to one of the eight possible combinations of the three input variables.

When enabled, exactly one output is active for a given input combination.

WHITEBOARD

Corresponds to Mano, Fig. 2-1.

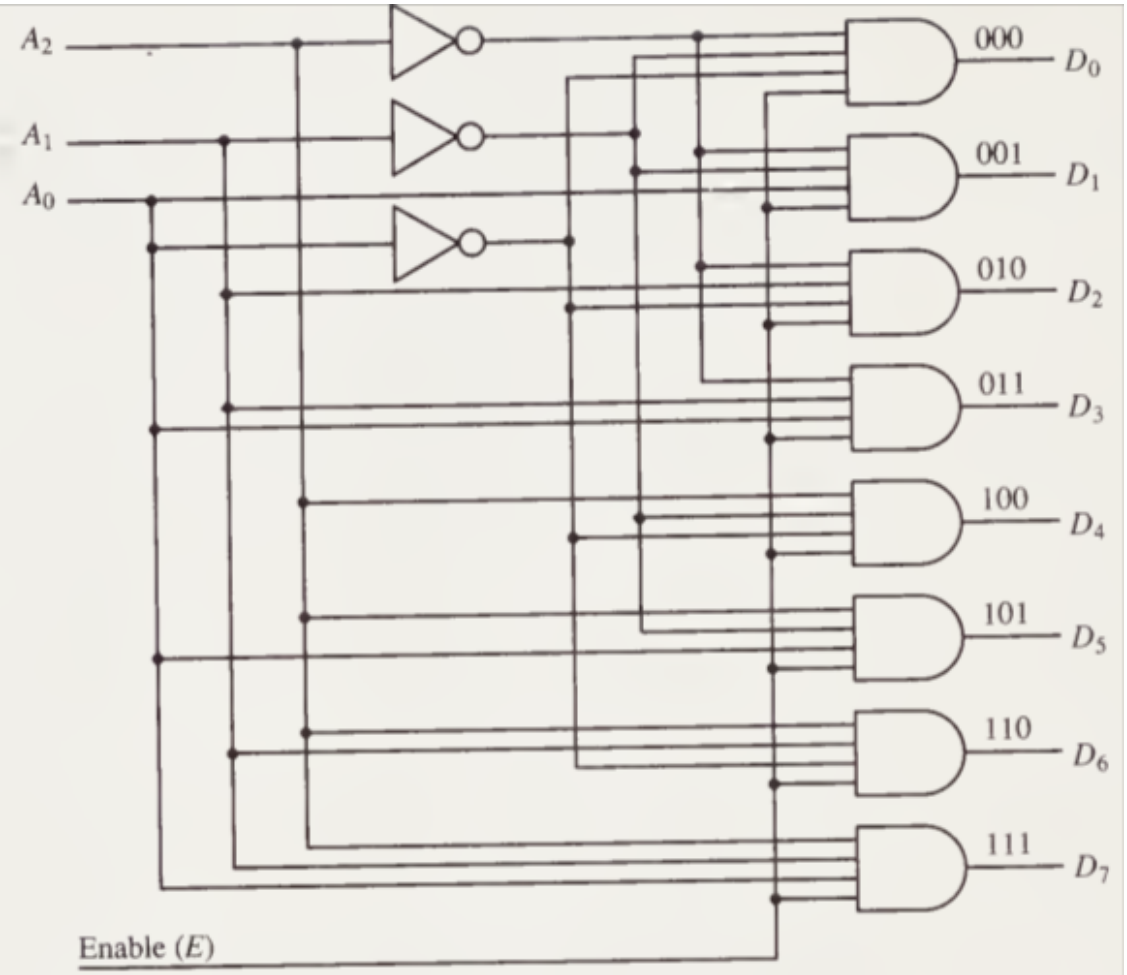


Figure 2-1 3-to-8-line decoder.

$$A_2A_1A_0 \rightarrow D_0, D_1, \ldots, D_7$$

one output corresponds to each 3-bit combination

The decoder can also be interpreted as a binary-to-octal converter.

For example,

$$A_2A_1A_0 = 110$$

selects output

$$D_6.$$

WHITEBOARD

$$A_2A_1A_0 = 110_2 = 6_{10}$$

$$\boxed{D_6 = 1}$$

All other decoder outputs:

$$D_i = 0 \qquad (i \neq 6)$$

2.2 Enable Input

Commercial decoders normally contain one or more enable inputs.

For the decoder of Fig. 2-1, the circuit is enabled when

$$E = 1.$$

When

$$E = 0,$$

all outputs are 0 regardless of the values of the three data inputs.

WHITEBOARD

$$\boxed{E = 0 \Rightarrow D_0 = D_1 = \cdots = D_7 = 0}$$

$$\boxed{E = 1 \Rightarrow \text{normal decoding}}$$

When enabled:

$$\boxed{\text{exactly one output} = 1}$$

2.3 NAND-Gate Decoder and Active-Low Logic

A decoder may also be constructed using NAND gates.

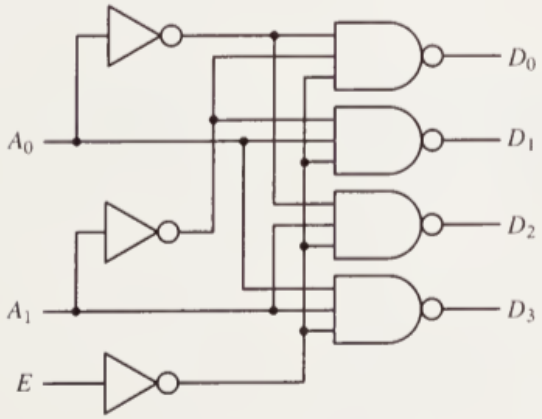
In this arrangement the outputs may be generated in complemented form.

The decoder of Fig. 2-2 is enabled when its complemented enable input is 0. When enabled, exactly one output is 0 and the remaining outputs are 1.

WHITEBOARD

Corresponds to Mano, Fig. 2-2.

Figure 2-2 2-to-4-line decoder with NAND gates.



(a) Logic diagram

E	A_1	A_0	D_0	D_1	D_2	D_3
0	0	0	0	1	1	1
0	0	1	1	0	1	1
0	1	0	1	1	0	1
0	1	1	1	1	1	0
1	×	×	1	1	1	1

(b) Truth table

active-high signal: asserted = 1

active-low signal: asserted = 0

For this decoder:

$\bar{E} = 0 \Rightarrow$ enabled

exactly one output = 0

2.4 Decoder Expansion

Enable inputs make it possible to combine smaller decoders into a larger decoder.

Mano demonstrates this by constructing a 3-to-8-line decoder from two 2-to-4-line decoders.

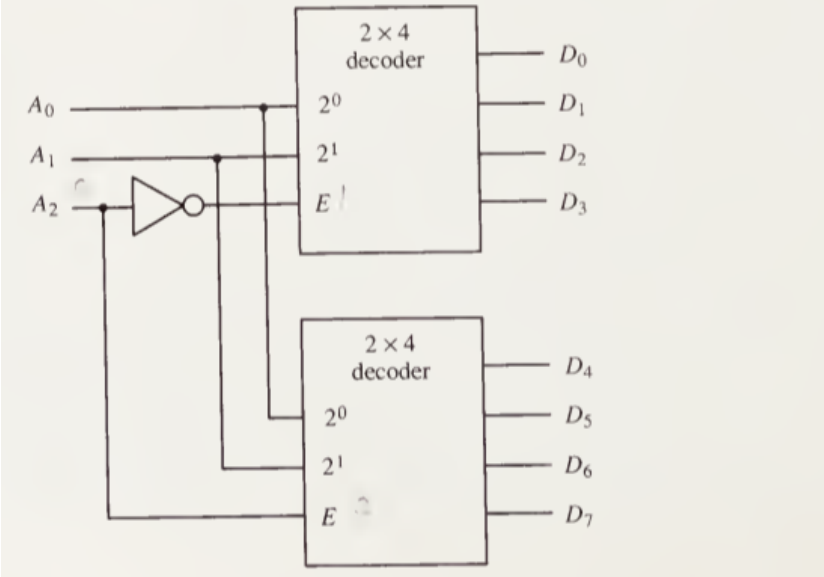
The two least significant input bits are applied to both smaller decoders.

The most significant bit determines which decoder is enabled.

WHITEBOARD

Corresponds to Mano, Fig. 2-3.

Figure 2-3 A 3 × 8 decoder constructed with two 2 × 4 decoders.



$A_1, A_0 \rightarrow$ both 2-to-4 decoders

$A_2 = 0 \Rightarrow D_0$–D_3

$A_2 = 1 \Rightarrow D_4$–D_7

Enable inputs allow component expansion

2.5 Encoders

An encoder performs the inverse operation of a decoder.

A decoder converts a binary code into one of many outputs.

An encoder converts one active input line into a binary code.

WHITEBOARD

Decoder: code → one-of-many

Encoder: one-of-many → code

For an octal-to-binary encoder, eight input lines correspond to the octal digits 0 through 7 and three output lines produce the equivalent binary code.

The circuit assumes that only one input is active at a time.

The output equations are

$$A_0 = D_1 + D_3 + D_5 + D_7,$$

$$A_1 = D_2 + D_3 + D_6 + D_7,$$

$$A_2 = D_4 + D_5 + D_6 + D_7.$$

WHITEBOARD

8-to-3 encoder

Only one input is assumed to be 1

$$A_0 = D_1 + D_3 + D_5 + D_7$$

$$A_1 = D_2 + D_3 + D_6 + D_7$$

$$A_2 = D_4 + D_5 + D_6 + D_7$$

---

3 Multiplexers

A multiplexer is a combinational circuit that selects binary information from one of several data input lines and directs the selected value to a single output.

A 2^n-to-1 multiplexer contains

$$2^n$$

data input lines and

n

selection inputs.

The multiplexer is therefore also called a *data selector*.

WHITEBOARD

$\boxed{2^n\text{-to-1 MUX}}$

2^n data inputs

n selection inputs

1 output

$\boxed{\text{MUX} = \text{data selector}}$

3.1 4-to-1 Multiplexer

A 4-to-1 multiplexer has four data inputs,

$$I_0, I_1, I_2, I_3,$$

two selection inputs,

$$S_1, S_0,$$

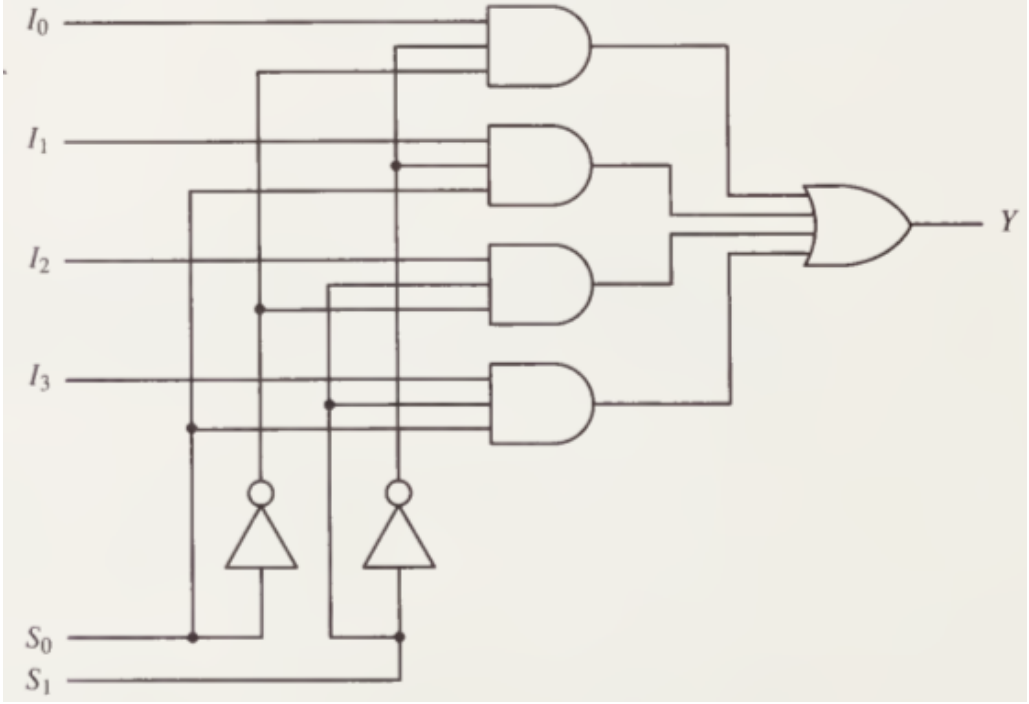
and one output,

$$Y.$$

WHITEBOARD

Corresponds to Mano, Fig. 2-4.

Figure 2-4 4-to-1-line multiplexer.



S_1	S_0	Y
0	0	I_0
0	1	I_1
1	0	I_2
1	1	I_3

For example,

$$S_1 S_0 = 10$$

selects input

$$I_2.$$

Thus,

$$Y = I_2.$$

WHITEBOARD

$$S_1 S_0 = 10$$

$$\boxed{Y = I_2}$$

selection lines determine which input reaches Y

---

4 Registers

A register is a group of flip-flops used to store binary information.

Since each flip-flop stores one bit, an n-bit register contains n flip-flops and can store one n-bit binary quantity.

WHITEBOARD

---

n-bit register = n flip-flops

1 flip-flop = 1 stored bit

register stores one binary word

4.1 4-Bit Register

The simplest register consists of flip-flops sharing a common clock.

In Mano's 4-bit register, four D flip-flops store four input bits.

At the active clock transition, all four inputs are transferred into the register simultaneously.

This operation is called a *parallel load*.

WHITEBOARD

Corresponds to Mano, Fig. 2-6.

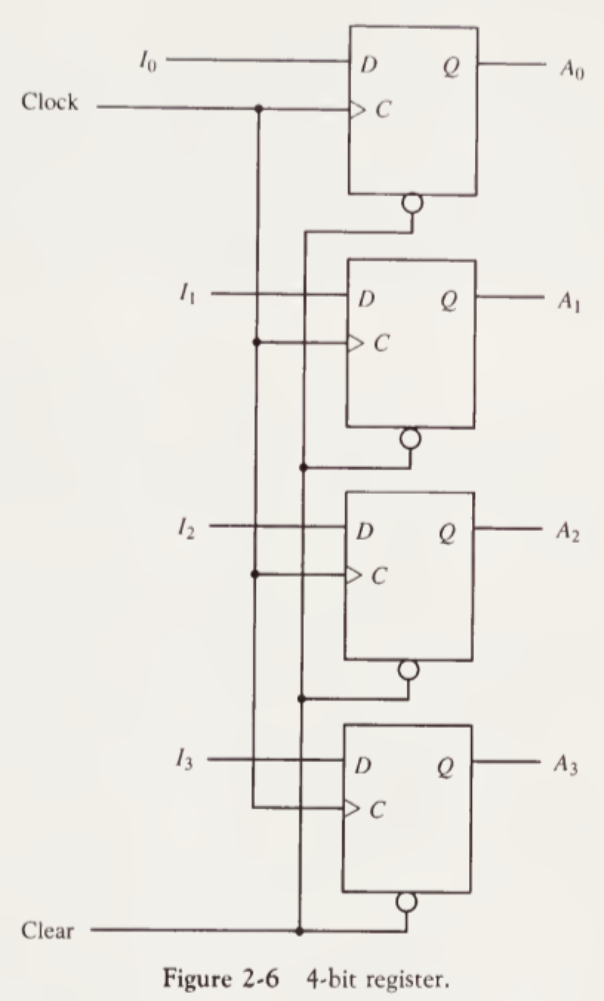


Figure 2-6 4-bit register.

$$I_0, I_1, I_2, I_3 \rightarrow A_0, A_1, A_2, A_3$$

all bits load simultaneously

simultaneous loading = parallel load

The clear input resets all flip-flops asynchronously.

During normal clocked operation, the clear input remains inactive.

4.2 Register with Parallel Load Control

A computer normally has a continuously running master clock.

A separate control signal is therefore needed to determine whether a particular register should accept new information.

In the register of Fig. 2-7, the clock is continuously available while the Load input determines what appears at each D input.

When

$$\text{Load} = 1,$$

the external input is selected.

When

$$\text{Load} = 0,$$

the current register output is fed back to the D input so that the state is preserved.

WHITEBOARD

Corresponds to Mano, Fig. 2-7.

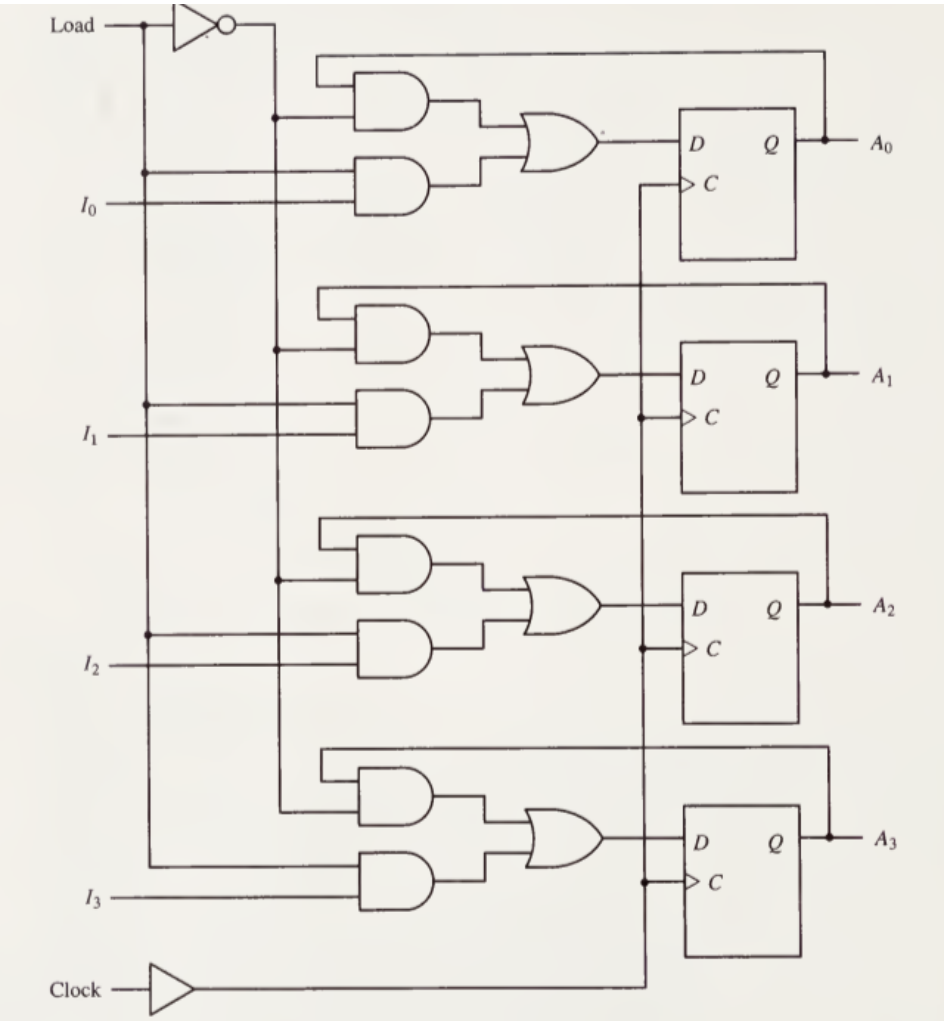


Figure 2-7 4-bit register with parallel load.

$$\boxed{\text{Load} = 1 \Rightarrow D_i = I_i}$$

$$\boxed{\text{Load} = 0 \Rightarrow D_i = A_i}$$

$$\boxed{\text{feedback provides the hold condition}}$$

The feedback path is necessary because the D flip-flop itself has no separate "no change" input combination.

5 Shift Registers

A shift register is a register capable of shifting its binary information from one stage to the next.

The output of each flip-flop is connected to the D input of the following flip-flop.

All stages share a common clock.

WHITEBOARD

$$\boxed{\text{Shift register} = \text{register that moves stored bits}}$$

$$\text{Serial In} \rightarrow A_0 \rightarrow A_1 \rightarrow A_2 \rightarrow A_3 \rightarrow \text{Serial Out}$$

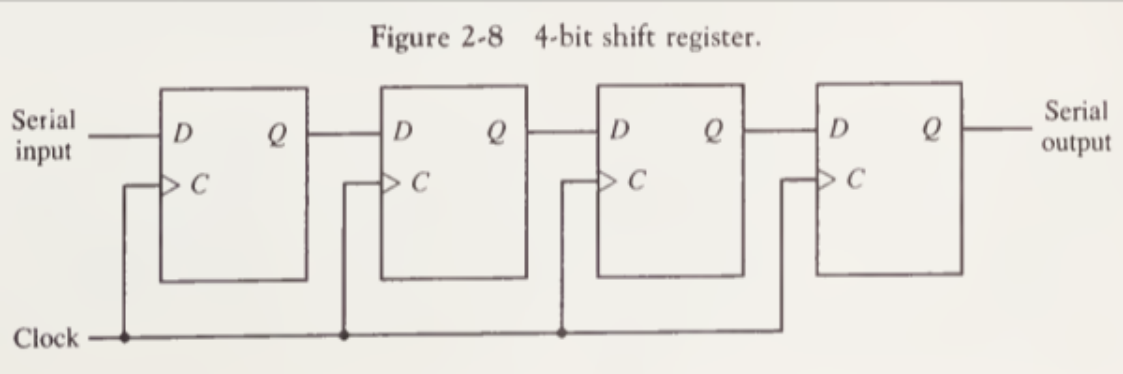
$$\boxed{1 \text{ clock transition} \Rightarrow 1 \text{ shift}}$$

5.1 4-Bit Shift Register

In the simple 4-bit shift register, the serial input enters the leftmost flip-flop and each stored bit moves one position toward the serial output on the active clock transition.

WHITEBOARD

Corresponds to Mano, Fig. 2-8.



$$\boxed{\text{Serial In} \rightarrow A_0 \rightarrow A_1 \rightarrow A_2 \rightarrow A_3 \rightarrow \text{Serial Out}}$$

5.2 Serial and Parallel Transfer

Parallel transfer moves multiple bits simultaneously over multiple data lines.

Serial transfer moves one bit at a time over a single data line.

Shift registers can therefore provide conversion between serial and parallel representations.

WHITEBOARD

Parallel transfer: n bits at once on n lines

Serial transfer: 1 bit at a time on 1 line

Transmitter:

parallel $\rightarrow$ serial

Receiver:

serial $\rightarrow$ parallel

5.3 Bidirectional Shift Register with Parallel Load

A more general shift register can:

- hold its present contents,
- shift right,
- shift left,
- load data in parallel.

Mano's 4-bit design uses one 4-to-1 multiplexer for each D flip-flop.

The selection lines S_1 and S_0 choose which data source reaches each flip-flop input.

WHITEBOARD

Corresponds to Mano, Fig. 2-9.

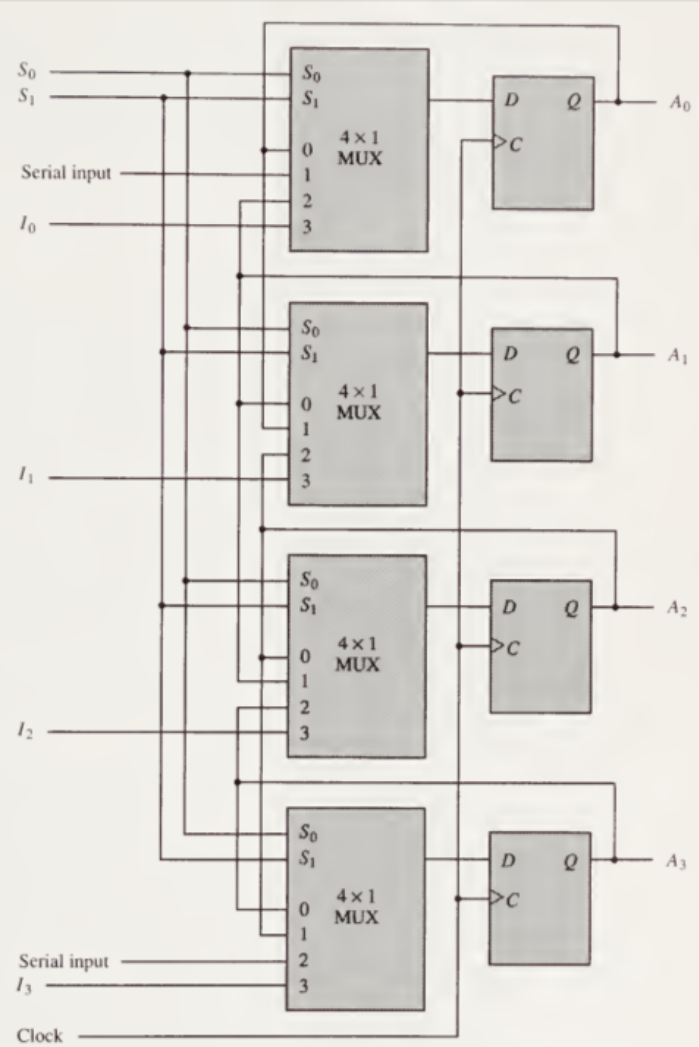


Figure 2-9 Bidirectional shift register with parallel load.

S_1	S_0	Register Operation
0	0	No change
0	1	Shift right
1	0	Shift left
1	1	Parallel load

The four multiplexer selections have a direct interpretation:

$$00 \Rightarrow \text{feedback},$$

$$01 \Rightarrow \text{shift-right source},$$

$10 \Rightarrow$ shift-left source,

$11 \Rightarrow$ parallel input.

WHITEBOARD

$\boxed{00 \Rightarrow \text{hold}}$

$\boxed{01 \Rightarrow \text{shift right}}$

$\boxed{10 \Rightarrow \text{shift left}}$

$\boxed{11 \Rightarrow \text{parallel load}}$

---

6 Binary Counters

A counter is a register that passes through a predetermined sequence of states when input pulses are applied.

A binary counter follows the ordinary binary number sequence.

An n-bit binary counter contains n flip-flops and counts from

$$0$$

through

$$2^n - 1.$$

WHITEBOARD

$\boxed{\text{Counter} = \text{register that follows a state sequence}}$

$\boxed{n\text{-bit binary counter: } 0 \rightarrow 2^n - 1}$

Example:

$$0000 \rightarrow 0001 \rightarrow 0010 \rightarrow 0011 \rightarrow 0100 \rightarrow \cdots$$

Inspection of the binary counting sequence gives a regular toggle pattern.

The least significant bit changes after every count.

Each higher-order bit changes only when all lower-order bits are 1.

WHITEBOARD

$\boxed{A_0 \text{ toggles every count}}$

$\boxed{A_i \text{ toggles iff all lower-order bits are 1}}$

Example:

$$0111 \rightarrow 1000$$

All four bits change because:

$$A_0 = 1, \qquad A_1A_0 = 11, \qquad A_2A_1A_0 = 111.$$

6.1 4-Bit Synchronous Binary Counter

JK flip-flops are convenient for binary counters because they toggle when

$$J = K = 1.$$

In a synchronous counter, every flip-flop receives the same clock.

Combinational logic determines which stages are permitted to toggle on each clock transition.

WHITEBOARD

Corresponds to Mano, Fig. 2-10.

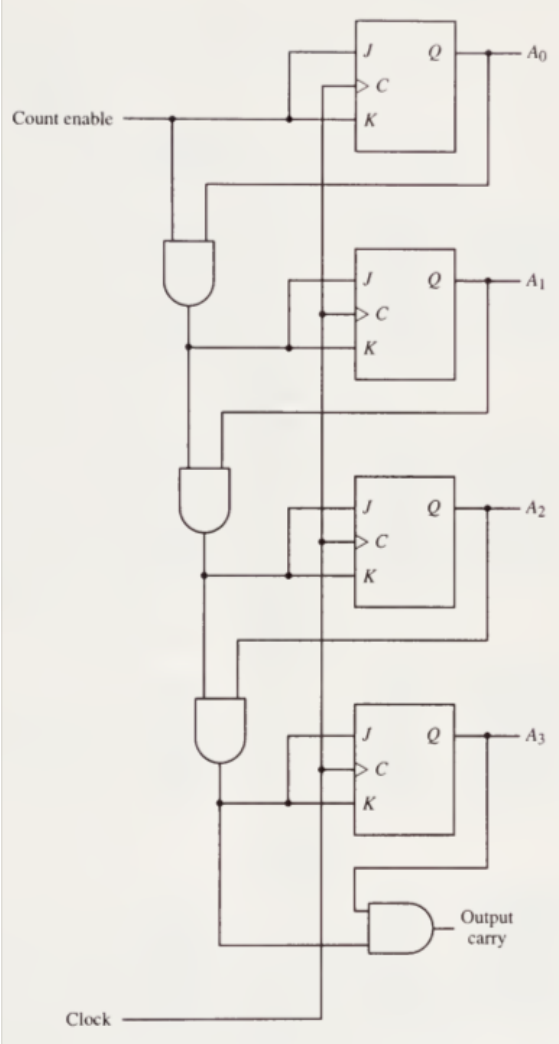


Figure 2-10 4-bit synchronous binary counter.

$$\text{Enable} = 0 \Rightarrow \boxed{\text{hold}}$$

$$\text{Enable} = 1 \Rightarrow \boxed{\text{count}}$$

$$A_0: \quad J_0 = K_0 = \text{Enable}$$

Higher stages toggle when:

$$\boxed{\text{Enable} = 1 \text{ and all lower-order bits} = 1}$$

The output carry is generated when all stages reach the condition required to advance an additional counter stage.

It can therefore be used to expand the counter.

6.2 Binary Counter with Parallel Load

Counters used in digital systems often require more operations than simple counting.

Mano's counter with parallel load provides:

- no change,
- increment,
- parallel load,
- synchronous clear.

WHITEBOARD

Corresponds to Mano, Fig. 2-11.

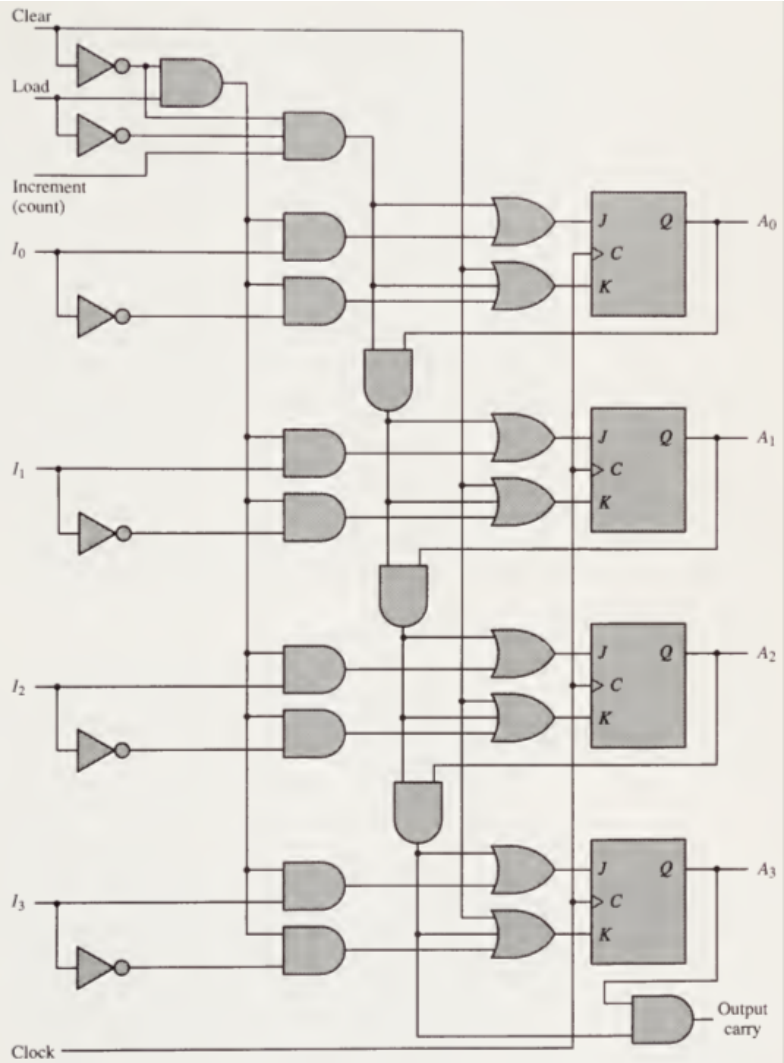


Figure 2-11 4-bit binary counter with parallel load and synchronous clear.

Clear	Load	Increment	Operation
0	0	0	No change
0	0	1	Increment by 1
0	1	X	Load I_0–I_3
1	X	X	Clear to 0

The increment operation adds one to the binary number stored in the register.

WHITEBOARD

$$\boxed{\text{Increment} = \text{register contents} + 1}$$

Priority:

$$\boxed{\text{Clear} > \text{Load} > \text{Increment} > \text{Hold}}$$

---

7 Memory Unit

A memory unit is a collection of storage cells together with the circuitry required to transfer information into and out of storage.

Binary information is stored in groups called *words*.

A word is transferred as one unit.

A group of eight bits is called a *byte*.

WHITEBOARD

$$\boxed{\text{word} = \text{group of bits stored/transferred as one unit}}$$

$$\boxed{1 \text{ byte} = 8 \text{ bits}}$$

Examples:

$$\text{16-bit word} = 2 \text{ bytes}$$

$$\text{32-bit word} = 4 \text{ bytes}$$

7.1 Memory Addressing

Each word in memory has a unique address.

If a memory contains

$$2^k$$

words, then

$$k$$

address lines are required to select one word.

If each word contains n bits, the memory organization can be written as

$$2^k \times n.$$

WHITEBOARD

$$\boxed{2^k \text{ words} \Rightarrow k \text{ address lines}}$$

$$\boxed{\text{memory organization} = 2^k \text{ words} \times n \text{ bits/word}}$$

Address range:

$$0 \rightarrow 2^k - 1$$

Memory sizes are commonly expressed using powers of two:

$$K = 2^{10},$$

$$M = 2^{20},$$

$$G = 2^{30}.$$

WHITEBOARD

$$\boxed{K = 2^{10}} \qquad \boxed{M = 2^{20}} \qquad \boxed{G = 2^{30}}$$

Examples:

$$64K = 2^{16}$$

$$2M = 2^{21}$$

$$4G = 2^{32}$$

---

8 Random-Access Memory

Random-access memory, or RAM, permits information to be transferred to or from any selected memory word.

The word is selected by its binary address.

A RAM unit therefore contains:

- address lines,
- data input lines,
- data output lines,
- read and write control lines.

WHITEBOARD

Corresponds to Mano, Fig. 2-12.

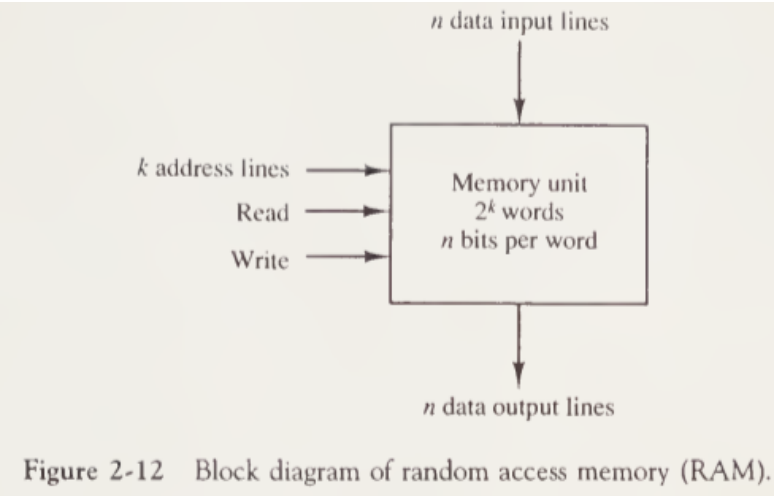


Figure 2-12 Block diagram of random access memory (RAM).

$\boxed{2^k \text{ words} \times n \text{ bits/word}}$

k address lines

n data input lines

n data output lines

Read, Write

8.1 Write Operation

To store a new word in RAM:

1. Apply the desired binary address.
2. Apply the data to the input lines.
3. Activate the Write control.

WHITEBOARD

$\boxed{\text{RAM Write}}$

1. Address

2. Data In

3. Write

$\boxed{\text{selected word} \leftarrow \text{input data}}$

8.2 Read Operation

To read a stored word:

1. Apply the desired binary address.
2. Activate the Read control.

The selected memory word is then placed on the output data lines.

Reading does not alter the stored word.

WHITEBOARD

RAM Read

1. Address

2. Read

3. Data Out

Read does not change stored data

---

9 Read-Only Memory

Read-only memory, or ROM, contains permanently established binary information.

A ROM has address inputs and data outputs but does not provide the normal write capability of RAM.

An $m \times n$ ROM contains

$$m$$

words of

$$n$$

bits each.

If

$$m = 2^k,$$

then k address lines are needed.

WHITEBOARD

Corresponds to Mano, Fig. 2-13.

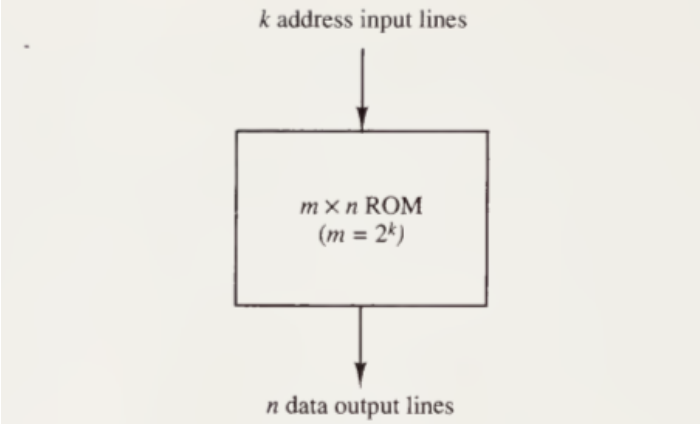


Figure 2-13 Block diagram of read only memory (ROM).

$$m = 2^k$$

$$\boxed{k \text{ address inputs} \rightarrow m \text{ stored words}}$$

$$\boxed{n \text{ output lines}}$$

A ROM output depends only on its current address inputs.

For this reason, Mano classifies ROM as a combinational circuit.

A ROM can implement an input-output relationship specified by a truth table.

WHITEBOARD

$$\boxed{\text{ROM output} = f(\text{address inputs})}$$

Therefore:

$$\boxed{\text{ROM is a combinational circuit}}$$

ROM can implement a truth table

Internal idea:

decoder + OR circuitry

9.1 Types of ROM

Mano describes several ways in which ROM information may be established.

WHITEBOARD

Types of ROM

ROM	fixed during manufacture
PROM	user-programmable once
EPROM	erasable and reprogrammable
EEPROM	electrically erasable

---

10 Chapter Summary

Chapter 2 moves from individual logic gates to reusable digital components.

Decoders and encoders convert between binary codes and one-of-many representations.

Multiplexers select one data source.

Registers store binary words.

Shift registers move stored information.

Counters advance through ordered binary states.

Memory units provide large collections of individually addressable words.

WHITEBOARD

---

gates → digital components

decoder → select one output

encoder → encode one active input

MUX → select one data input

register → store bits

shift register → move bits

counter → sequence through states

memory → store addressed words
